

Runta Product Trial - Final Feedback

Hritik Hassani
Product Engineering Trial

Summary

I treated this trial as a first-time developer evaluation rather than a feature checklist. I started with a small coding-agent task, then moved to a two-service A/B experiment, and finally built and deployed **RootScope**, an AI incident-investigation product that uses Runta for runtime isolation, tool execution, secrets, ingress, checkpoints, and Token X-Ray.

My overall impression is positive: the core runtime model is useful and the product has a strong fit for agent workloads. The best parts were reproducible runtime state, checkpoint-based branching, secret injection, egress controls, CLI/SDK interoperability, SSH, and Token X-Ray once the traffic path was configured correctly.

The biggest friction was not that features were missing, but that several important states were difficult to observe. In a few places a feature looked enabled while the effective behavior was different: Token X-Ray capture after CLI checkpoint restore, provider/model selection, custom image builds, and suspend/wake behavior. Those issues were only obvious after measuring the result independently.

What I built

1. Repo Doctor

I started with a deliberately buggy Python repository where `divide(a, b)` multiplied instead of dividing. I established the failing baseline locally and in Runta, checkpointed the prepared runtime, restored identical branches, and compared two coding-agent strategies.

The important part of this first experiment was the method: I did not accept the agent's own statement that it had fixed the code. I independently checked the git diff and ran pytest from outside the agent flow.

This experiment also led to the first Token X-Ray investigation and several lifecycle, SSH, egress, SDK, and custom-image tests.

2. Microservice Agent Lab

I then built a small FastAPI system with two real communicating services:

- inventory on port 8001
- orders on port 8002
- a published inventory contract
- unit, contract, and end-to-end tests

The task was to rename `available` to `available_quantity` without breaking the consumer.

I used one checkpoint as the identical starting state and compared:

- one repo-wide agent
- two service-scoped agents coordinating through the contract

Both approaches passed the independent test suite: **8 unit/contract tests + 4 integration tests**.

The repo-wide agent was more token-efficient once both sides were captured: **14 calls / 6,173 estimated tokens** versus **26 calls / 13,005 estimated tokens** for the two scoped agents. The scoped path produced the cleaner boundary: the producer changed first, the system was observably broken at the service contract, and the consumer was then migrated from the published contract. The global agent changed more files, including documentation and a contract version bump.

I would choose the global agent for lower duplicated exploration cost, and the scoped model when service ownership and a small blast radius matter more than token efficiency.

3. RootScope

The final product is **RootScope - AI-powered incident investigation for software systems**.

It has:

- Next.js frontend
- FastAPI backend
- OpenAI Responses API agent
- tool-driven investigation over files, logs, services, tests, shell commands, and git diff
- inventory + orders demo services
- three reproducible incidents: contract regression, configuration failure, and noisy production logs
- SSE investigation activity
- structured diagnosis and evidence
- sandboxed Apply Fix
- test validation and git diff
- Change Brief, Runbook, Postmortem, and conditional Decision Brief
- SQLite persistence

The final Python test suite is **45/45 passing**.

On Runta I deployed RootScope on a **4 vCPU / 8 GiB** runtime with:

- `openai_responses`
- `explicit gpt-4.1`

- OpenAI key from Runta Secrets
- Token X-Ray enabled
- all four token-saving policies enabled
- only port 3000 published

The contract incident was diagnosed correctly. After the later noisy-log investigation, Token X-Ray on the same runtime showed a cumulative **17 calls / 5,839 estimated tokens**, up from **6 calls / 841 tokens** after the earlier contract run. The noisy-log agent searched for the buried error rather than dumping the entire 4,000-line log, which was a good agent behavior. WatchFox instead flagged a large `git_diff` tool output of roughly 16K characters / 4,004 estimated tokens.

I also created a RootScope checkpoint and restored it through the REST API while explicitly reapplying capture, secret, ingress, and token-saving policy. The restored runtime came up with all of those settings present.

What worked well

Checkpoints and runtime state

This was the strongest part of the product for me. Checkpoints made controlled agent comparisons practical because I could start multiple experiments from the same prepared state.

In the workflows I tested, filesystem state, installed dependencies, repository state, and observed agent authentication survived checkpoint/restore. Pause/resume also preserved the working state and uncommitted files.

Secrets and egress

Runta Secrets worked well for the OpenAI key. I could inject `OPENAI_API_KEY` and add the Authorization header only for `api.openai.com/v1/*` without putting the key in source.

Allowlist and denylist tests also behaved as expected. This is especially useful for agent runtimes where I want the code to have broad local execution capability but controlled external access.

CLI, SDK, and SSH interoperability

The Python SDK could create, list, pause, resume, and delete runtimes, and those runtimes were visible through the CLI. SSH also worked through Runta's command, an explicit identity, and generated native SSH config. State persisted across pause/resume.

Token X-Ray / WatchFox when the path was correct

Once I used an explicit OpenAI API-key path and model, capture was useful. It detected repeated calls, repeated paths, and large outputs.

In a targeted large-log test, WatchFox recorded **60 actual tokens saved** by the log optimization. I do not attribute the full difference between model runs to compression; only the 60 tokens explicitly reported by Runta are counted as savings.

Public ingress

The port-prefixed ingress URL worked while the runtime was running. RootScope publishes only Next.js on port 3000, while FastAPI and the demo services stay inside the runtime and are reached through the frontend proxy.

Friction and issues I found

1. CLI checkpoint restore dropped Token X-Ray capture

This was the most important observability issue in the trial.

A base runtime had `llm_tool_io_capture_enabled: true`. After restoring the checkpoint with the CLI, the restored runtime had capture disabled. The coding agent then ran normally, but `runta tokens analyze` showed **0 calls / 0 tokens**.

When I created the restored runtime through `POST /v2/runtimes` with the same checkpoint and explicitly set `llm_tool_io_capture_enabled: true`, capture worked.

Impact: a user can believe they are running an instrumented A/B experiment while silently losing the observability needed to compare the runs.

Suggestion: preserve capture policy on CLI restore, add capture flags to the restore command, or explicitly warn that observability policy must be reapplied.

2. “Capture enabled” did not mean “provider traffic is being captured”

Early Token X-Ray tests used the subscription-based runtime sign-in path. The runtime showed capture enabled, but WatchFox stayed at 0/0 because that traffic was not going through the OpenAI Responses API path being observed.

Later API-key runtimes captured correctly.

Suggestion: show a second status such as **“Capture enabled - 0 supported provider requests observed”**. That would have saved a lot of debugging time.

3. Model/provider compatibility needed explicit configuration

The default path selected an incompatible model in my testing. On the final RootScope deployment I had to create the runtime through the REST API and explicitly set `model_provider_model: gpt-4.1`.

The installed CLI exposed `--model-provider-protocol` but did not expose the equivalent explicit model field I needed.

Suggestion: validate model/provider compatibility at create time and keep CLI/API capabilities aligned.

4. Docker worked inside the runtime, but nested builds needed the Runta CA

The Docker daemon and Compose worked inside a Runta runtime. A Docker build could pull the base image and complete apt operations, but `pip install` inside the child image failed TLS verification because the child container did not inherit Runta's egress CA.

The host had `/usr/local/share/ca-certificates/runta-egress.crt` and could reach PyPI. Copying that certificate into the child image and running `update-ca-certificates` made the exact Docker build succeed.

Suggestion: document Docker-in-Runtime CA propagation or provide an easier supported mechanism for child build environments.

5. Custom Runtime Images were the least reliable surface I tested

I saw two different failure stages:

1. Earlier builds started backend-managed attempts, retried, and ended with `cloud_build_staging_failed` / **"Build files could not be transferred. Try again."**
2. Later clean builds were accepted but remained pending, attempt: 0, with no first attempt starting.

The same Dockerfile built successfully locally for `linux/amd64`, and a version also built successfully with Docker inside a Runta runtime after adding the egress CA.

During the RootScope pass, a fresh image build did at least advance to building and reached attempt 3 in the last captured state, but the UI/API still did not expose enough build-stage logs to explain what the worker was doing.

Suggestion: expose build phases, worker assignment, per-attempt logs, staging errors, and a dedicated image-build status indicator.

6. Auto-suspend worked; wake behavior did not match my expectation

Fresh runtimes did auto-suspend at the configured timeout. In my tests, however:

- `runta exec` against a suspended runtime repeatedly returned HTTP 500 and did not wake it
- dashboard terminal access did not wake it
- HTTP traffic to a published service did not trigger the wake transition I expected
- manually resumed runtimes could re-suspend quickly

I could always use `runta resume`, but the product/documentation made the wake behavior look more automatic than what I observed.

Suggestion: make wake semantics explicit per activity type and return an actionable suspended-state error rather than a raw 500.

7. Documentation and installed CLI drifted in several places

Examples from the trial included:

- agent command examples that did not exist in CLI 0.2.4
- SSH examples using older syntax
- `api.runta.me` in documentation while the valid key in my tests was accepted by `api.runta.com`
- inconsistent public ingress hostname examples
- `runta` tokens being important for debugging capture but not easy to discover from the Token X-Ray docs

None of these individually blocked the trial, but together they increased the amount of time spent distinguishing product behavior from version drift.

Suggestion: version docs with the CLI or generate command/API reference from the same schema where possible.

8. Quota state was easy to discover only after creation failed

The first 4 vCPU RootScope create hit `resource_limit_exceeded` because older paused/suspended trial runtimes still counted toward the trial organization's shared vCPU limit. After deleting old runtimes, the same deployment could proceed.

Suggestion: show reserved-vCPU / memory usage and remaining org capacity prominently before create/restore.

Token X-Ray observations

Token X-Ray was most useful when I treated it as an engineering signal rather than a score.

The microservice comparison showed that a scoped multi-agent workflow used more tokens because each agent re-explored its part of the repository. That is expected and useful information, not necessarily a defect.

RootScope's noisy-log incident also showed an important positive behavior: the agent used a targeted ERROR query and returned one matching line instead of reading all 4,000 lines. The largest output in that run was actually an unconstrained git diff, which WatchFox flagged correctly.

That suggests a useful product direction: combine findings with specific guidance such as **"scope this git diff to the service directory"** or **"this tool returned 4K estimated tokens; consider a query/path filter"**.

What I would prioritize

If I were working on the product, my order would be:

1. **Make effective runtime state observable.** Show whether provider traffic is actually flowing through capture, not only whether the flag is enabled.

2. **Make checkpoint restore policy-safe.** Restore or explicitly reapply X-Ray, secret, ingress, and token-saving policy.
3. **Improve custom-image diagnostics.** Build logs and stage visibility would turn a black-box failure into something users can self-debug.
4. **Clarify suspend/wake semantics.** Treat suspended as a first-class lifecycle state in CLI errors and docs.
5. **Unify CLI/API/docs capabilities.** Especially model selection, restore options, API hostnames, and ingress examples.
6. **Document nested-container trust.** Agents increasingly run Docker/Compose, so CA propagation matters in real workloads.

Final assessment

Runta was useful for the workflows I care about: reproducible agent environments, checkpoints, isolated execution, secrets, controlled egress, and tool-call observability. I was able to build real experiments on top of it rather than only run hello-world commands.

The product's biggest opportunity is making hidden runtime policy and backend state visible. When Runta worked, it made difficult agent experiments much easier to reproduce. When it did not, the hardest part was often determining whether the issue was my code, the agent, the runtime policy, or a backend worker.

That is also why the trial was useful: the core abstraction felt strong enough that the rough edges were worth debugging rather than reasons to abandon the workflow.